Assistente de mobilidade por ônibus para pessoas cegas e com baixa visão

Aplicativo + dispositivo vestível (ESP32-CAM) que identifica o ônibus certo e acompanha o trajeto

Documento de projeto — prova de conceito e caminho para produto físico e software para óculos inteligentes

Versão 0.1 · 11 de novembro de 2026

1. Resumo executivo

O projeto propõe um sistema de mobilidade urbana para pessoas cegas ou com baixa visão que usam ônibus. O usuário define no aplicativo a rota e as linhas que pretende pegar. Um pequeno dispositivo com **ESP32 e câmera**, usado junto ao corpo, olha para o ônibus que se aproxima e envia as imagens ao celular por Wi-Fi (o celular atua como *hotspot*). O aplicativo identifica a linha exibida na placa do ônibus e, se for a desejada, avisa o usuário por **vibração no celular e áudio no fone de ouvido**. Durante a viagem, o aplicativo acompanha a posição por GPS e avisa quando se aproximar do ponto de descida.

A solução tem dois objetivos complementares:

- **Produto físico imediato:** um kit (dispositivo + app) para quem não tem óculos inteligentes.
- **Prova de conceito de software:** validar a lógica (reconhecimento da placa, rota, alertas e acompanhamento) que depois pode rodar em óculos de realidade aumentada com SDK/API aberta, sem depender do hardware próprio.

Modelo de negócio em duas frentes: **venda de software** (licença/assinatura) para quem já tem óculos compatíveis e **venda do hardware + app** para quem não tem.

2. Problema e público-alvo

Pessoas cegas ou com baixa visão têm dificuldade em identificar qual ônibus está chegando ao ponto, porque a linha e o destino aparecem em placas visuais (painéis de LED, letreiros). Hoje dependem de ajuda de terceiros, de perguntar ao motorista ou de outros passageiros, o que reduz a autonomia e pode causar atrasos ou riscos. Aplicativos de trânsito informam horários, mas **não resolvem o momento crítico: reconhecer o ônibus certo na hora em que ele para**.

Público principal: pessoas cegas ou com baixa visão que usam transporte público de forma regular. **Públicos secundários:** pessoas idosas, com dificuldade de leitura à distância, e turistas em cidades desconhecidas. **Parceiros naturais:** associações e instituições de apoio a pessoas com deficiência visual, órgãos de transporte e prefeituras.

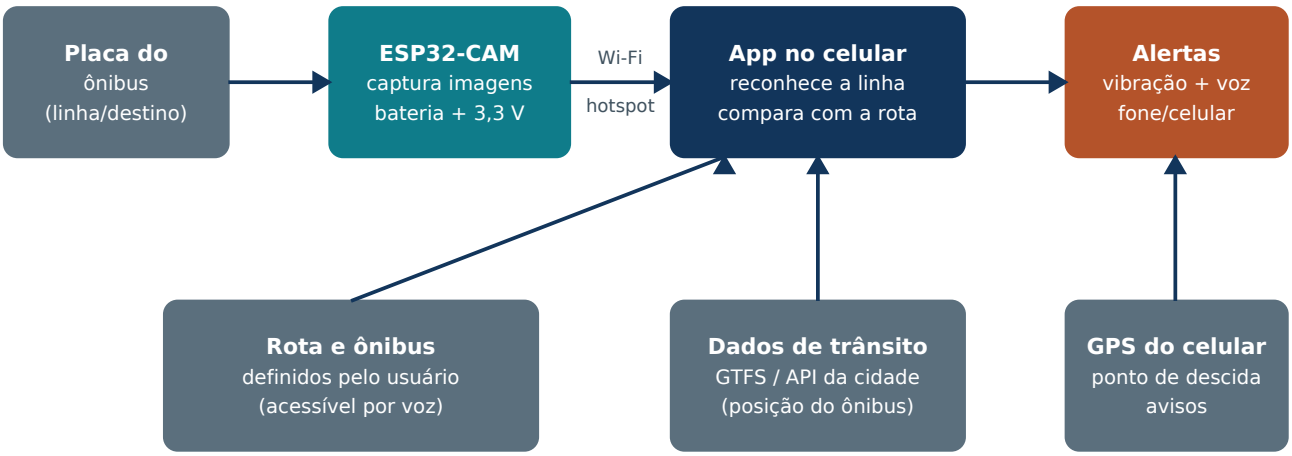
3. Visão geral da solução

Componente	Função
Aplicativo móvel	Cadastro de rota e linhas, comunicação com o dispositivo, reconhecimento da placa, alertas, acompanhamento por GPS, interface acessível por voz e leitor de tela.
Dispositivo ESP32-CAM	Captura imagens do ônibus e as envia ao celular. Alimentado por bateria com regulador de tensão. Usado preso à roupa, à bolsa, à bengala ou à armação de óculos.
Conexão	Wi-Fi: o ESP32 conecta-se ao hotspot do celular. Opcionalmente, BLE para pareamento inicial e comandos de baixo consumo.
Alertas	Vibração no celular e áudio (voz sintetizada e tons) no fone de ouvido.
Dados de trânsito	GPS do celular e, quando disponível, dados da cidade (GTFS / API de posição de ônibus) para confirmar linha e prever chegada.

4. Fluxo de uso

Etapa	O que acontece
1. Planejar	O usuário abre o app (por voz ou leitor de tela), informa origem e destino e escolhe as linhas de ônibus que vai pegar.
2. Conectar	Liga o dispositivo. Ele se conecta ao hotspot do celular e o app confirma por áudio: “câmera conectada”.
3. Aguardar no ponto	O app entra em modo de busca. A câmera captura imagens dos ônibus que chegam.
4. Identificar	O app reconhece a linha na placa. Se for uma linha da rota, avisa por vibração e voz (ex.: “ônibus 175T chegando”). Se não for, permanece em silêncio ou informa apenas se o usuário preferir.
5. Embarcar	O usuário sobe no ônibus. O app registra a viagem em andamento.
6. Acompanhar	Com GPS (e dados de trânsito), o app informa paradas restantes e avisa antes do ponto de descida.
7. Descer e seguir	Ao descer, o app continua a rota a pé ou indica a próxima linha, se houver baldeação.

5. Arquitetura do sistema



Fluxo: a câmera vê a placa → o celular decide se é o ônibus certo → o usuário é avisado.

Decisão de projeto central: **o processamento pesado fica no celular**. O ESP32 faz captura e transmissão; o reconhecimento da placa (OCR ou modelo de visão) roda no aplicativo. Isso reduz custo, consumo e complexidade do hardware e facilita evoluir o algoritmo por atualização de software. A mesma lógica poderá ser portada para óculos inteligentes, onde a câmera e o processamento já existem.

Modos de comunicação possíveis entre dispositivo e celular:

- **Transmissão de quadros JPEG por HTTP/WebSocket** (mais simples para a prova de conceito).
- **Envio apenas de quadros relevantes** (por exemplo, quando há movimento ou a cada N segundos) para economizar bateria.
- **BLE para controle** (ligar/desligar câmera, nível de bateria) e Wi-Fi apenas quando precisar de imagem.

6. Hardware

Lista inicial de componentes para o protótipo (preços a cotar):

Item	Observação
ESP32-CAM (ex.: módulo AI-Thinker, câmera OV2640)	Tem PSRAM e câmera integrada. Boa opção de baixo custo para a prova de conceito. Pode-se migrar depois para módulos ESP32-S3 com câmera, que têm mais desempenho.
Bateria Li-ion/LiPo 3,7 V (1 célula)	Capacidade a definir pelo consumo e autonomia desejada (ex.: 1000–2000 mAh para um primeiro teste).
Módulo de carga e proteção (ex.: TP4056 com proteção)	Carga por USB e proteção contra sobrecarga/descarga profunda.
Regulador de tensão para 3,3 V	Ver nota abaixo sobre a escolha entre LDO, buck e buck-boost.
Capacitores de desacoplamento	Ex.: 100 µF–470 µF próximos à alimentação do ESP32, para absorver picos de corrente do Wi-Fi e da câmera.
Chave liga/desliga e LED de estado	O LED usa resistor limitador. Opcionalmente, um divisor resistivo para ler a tensão da bateria no ADC.
Estojo e fixação	Caixa pequena com abertura para a câmera, clipe ou velcro. Proteção contra chuva em versões futuras.

Sobre alimentação (pontos de atenção). O ESP32 trabalha em 3,3 V, mas uma bateria de célula única Li-ion/LiPo fornece cerca de 4,2 V (cheia) a 3,0–3,3 V (descarregada), por isso o regulador é necessário. Um LDO de baixa queda (*low dropout*) é simples, mas perde a regulação quando a bateria chega perto de 3,3 V. Um conversor **buck-boost** mantém 3,3 V em toda a faixa e tende a ser a melhor escolha. Corrente de pico com Wi-Fi e câmera pode passar de 300 mA, então o regulador deve ter folga (a partir de 500 mA) e a fiação deve ser curta. Quedas de tensão são causa comum de reinicializações (*brownout*). Em geral, **não se usa resistor para adaptar a tensão de alimentação**: resistores entram em LED, pull-ups e divisor de tensão para medir a bateria. Vale validar esses valores no protótipo, medindo o consumo real.

7. Software (aplicativo)

Módulo	Descrição
Acessibilidade	Interface totalmente operável por leitor de tela (TalkBack/VoiceOver), com comandos de voz, alto contraste e fontes grandes para baixa visão.
Rotas e linhas	Cadastro de origem, destino e linhas. Integração com dados de trânsito da cidade (GTFS estático e, quando houver, tempo real).
Conexão com o dispositivo	Descoberta do ESP32 no hotspot, reconexão automática, indicador de bateria e de estado.
Reconhecimento da placa	Detecção do letreiro e leitura da linha/destino (OCR e/ou modelo de visão), com filtragem por confiança e votação em vários quadros.
Alertas	Vibração com padrões distintos (ônibus certo, ônibus errado, ponto de descida), voz sintetizada e tons no fone. Preferências configuráveis.
Acompanhamento a bordo	GPS do celular, paradas restantes, aviso antecipado do ponto de descida, tratamento de perda de sinal.
Privacidade	Processamento local, sem armazenar imagens por padrão, em conformidade com a LGPD.

8. Desafios técnicos e decisões em aberto

Tema	Risco / decisão
Leitura de painéis de LED	Painéis de LED piscam (PWM) e a câmera pode captar a placa com falhas ou faixas. Exige ajuste de exposição, múltiplos quadros e testes em condições reais (dia, noite, chuva).
Distância e ângulo	A câmera precisa ver a placa com resolução suficiente. A posição de uso (peito, bengala, óculos) e a lente afetam o resultado.
Latência	O ônibus fica pouco tempo parado. O ciclo captura → rede → reconhecimento → alerta precisa ser curto (meta de poucos segundos).
Alerta “por vibração no fone”	Fones comuns não vibram. A solução prática é áudio no fone e vibração no celular; vibração no próprio dispositivo ou em uma pulseira pode ser uma opção futura.
Hotspot	O hotspot consome bateria do celular e exige que o recurso esteja ativo. Avaliar BLE para controle e Wi-Fi sob demanda.
Posição do ônibus	Nem toda cidade oferece API de tempo real. O GPS do celular dentro do ônibus cobre o trecho a bordo, mas não a espera no ponto.
Segurança do usuário	O sistema é um apoio e não substitui bengala, cão-guia ou atenção ao entorno. Comunicar limites claramente e testar com usuários reais.
Imagens de terceiros	A câmera pode captar pessoas. Processar localmente, descartar quadros e informar a política de privacidade.

9. Da prova de conceito aos óculos inteligentes

Óculos inteligentes com câmera e áudio já reúnem o que o protótipo constrói separadamente: câmera na altura dos olhos, fone integrado e conexão com o celular. A prova de conceito com ESP32 valida o **núcleo de software**: reconhecimento da linha, comparação com a rota, alertas e acompanhamento do trajeto. Para facilitar a migração, o aplicativo deve separar claramente a **camada de entrada de imagem** (ESP32 hoje, óculos amanhã) da lógica de decisão.

- Definir uma interface única de “fonte de câmera” (quadros + metadados).
- Manter o reconhecimento e a lógica de rota independentes do hardware.
- Verificar, para cada plataforma de óculos, o que o SDK/API permite em termos de acesso à câmera, áudio e execução em segundo plano. As condições mudam com frequência e precisam ser confirmadas antes do desenvolvimento.

10. Modelo de negócio

Linha	Público	Receita
Software para óculos	Quem já tem óculos compatíveis (SDK/API aberta).	Assinatura mensal/anual ou licença única. Custo marginal baixo e escala fácil.
Kit de hardware + app	Quem não tem óculos inteligentes.	Venda do dispositivo, com app gratuito ou com assinatura opcional de recursos avançados.
Parcerias institucionais	Associações, escolas especializadas, órgãos de transporte, prefeituras.	Compras em lote, projetos financiados, licenças corporativas.

Vantagens da combinação: o kit físico cria uma base de usuários sem depender de óculos caros, gera dados reais para melhorar o reconhecimento e valida o produto; o software para óculos amplia o mercado quando essas plataformas se popularizarem. **Pontos a estudar:** custo de fabricação em pequena escala, certificações (Anatel, bateria), suporte ao cliente e acessibilidade da embalagem e do manual.

11. Roadmap sugerido

Fase	Entregas	Critério de sucesso
0. Preparação	Escolha da cidade piloto, levantamento das linhas e dos tipos de placa, contato com usuários e associações.	Lista de linhas-alvo e requisitos validados.
1. Prova de conceito de visão	ESP32-CAM transmitindo imagens ao celular; app de teste lendo a linha em fotos e vídeos de ônibus.	Taxa de acerto aceitável em um conjunto de testes reais.
2. Protótipo integrado	App com rota, conexão por hotspot, alertas por voz e vibração, bateria e regulador.	Fluxo completo no ponto de ônibus, com autonomia de algumas horas.
3. Acompanhamento a bordo	GPS, aviso de ponto de descida, integração com dados de trânsito.	Descida correta em viagens de teste.
4. Testes com usuários	Pilotos com pessoas cegas e com baixa visão; ajustes de uso, ergonomia e interface.	Feedback positivo e métricas de autonomia.
5. Produto e óculos	Versão fabricável do dispositivo, app publicado, porte para óculos com SDK aberto.	Primeiras vendas e piloto em óculos.

12. Próximos passos imediatos

- Montar o protótipo de bancada: ESP32-CAM, bateria, módulo de carga e regulador, e medir o consumo real.
- Coletar fotos e vídeos de placas de ônibus da cidade piloto em diferentes horários e distâncias.
- Testar OCR/modelos de visão no celular sobre esse conjunto e medir acerto e latência.
- Definir a cidade e a fonte de dados de trânsito (GTFS e/ou API em tempo real).
- Conversar com usuários e organizações de apoio para validar o fluxo e os alertas.
- Pesquisar as condições atuais de SDK/API de óculos inteligentes candidatos.

Este documento é um ponto de partida. Os valores de capacidade de bateria, corrente e desempenho indicados são estimativas a validar no protótipo.